

SQL ReAct Agent

Natural Language to SQL Querying using the ReAct Framework

Project Repository Link: [SQL-ReAct-Project](#)

Submitted By:

Veer Shah (2022300107)
Muskan Sharma (2022300111)

Department of Computer Engineering
Sardar Patel Institute of Technology, Mumbai

November 2025

Contents

1	System Overview	2
1.1	Purpose	2
1.2	Setup and Execution	2
1.2.1	Dependencies	2
1.2.2	API Configuration	2
1.2.3	Running Queries	2
1.3	Available Tools	3
1.4	Example Run and Trace	4
2	Test Results (6 Tests)	4
2.1	Test-1: Schema Discovery	4
2.2	Test-2: Simple Count	5
2.3	Test-3: Aggregation Query	5
2.4	Test-4: Filtering Query	6
2.5	Test-5: Customer count	6
2.6	Test-6: Error Recovery Case	7
3	Reflection	7
3.1	Design Tradeoffs	8
3.2	Observed Failure Modes	8
3.3	Future Enhancements	8
4	Conclusion	8

1 System Overview

1.1 Purpose

This project implements a fully custom SQL Database Agent using the ReAct (Reasoning + Acting) paradigm. Unlike existing frameworks such as LangChain or LangGraph, this implementation is entirely from scratch and focuses on explainability, safety, and correctness.

The system enables users to ask natural language questions about a SQLite database. It performs step-by-step reasoning, uses schema-aware decisions, and executes only safe, read-only SQL queries. Each agent decision is logged through a structured sequence of:

THOUGHT → ACTION → OBSERVATION → FINAL ANSWER

allowing transparent insight into how the model arrives at each result.

Core goals:

- **Interpret natural language questions**
- **Explore and understand database schema automatically**
- **Construct safe SQL queries**
- **Execute read-only database interactions**
- **Provide full reasoning trace for educational and debugging purposes**

1.2 Setup and Execution

1.2.1 Dependencies

The implementation is intentionally lightweight, depending only on:

- **google-generativeai** (Gemini API)
- Python 3.10+
- Standard libraries (**sqlite3, json, logging, time, etc.**)

1.2.2 API Configuration

Before running, set your API key in the script:

```
1 GEMINI_API_KEY = "YOUR_API_KEY"
```

Listing 1: API Configuration

1.2.3 Running Queries

- **Single Query Mode:** Allows the user to ask one question directly from the terminal

```
1 python single_query.py "What is the total amount of all orders?"
```

Listing 2: Single Query Mode

- **Test Suite Execution:** A complete set of 6 automated tests can be run using

```
1 python test_agent.py
```

Listing 3: test Suite Execution

The test suite covers:

- Schema Discovery
- Counting queries
- Aggregations
- Filtering
- Customer Metadata Queries
- Error Detection and Tool Recovery

Rate-limiting delays and automatic retries are integrated to maintain stable execution.

1.3 Available Tools

The agent uses only three tools, enabling strict control and preventing misuse:

Tool Name	Arguments	Returns	Description
<code>list_tables</code>	None	List of table names	Discovers all tables present in the SQLite database and returns them as a list.
<code>describe_table</code>	<i>table_name</i> (string)	Schema: columns, datatypes, row count	Retrieves the full schema of a given table, including its columns, their types, and the number of rows in the table.
<code>query_database</code>	<i>query</i> (SELECT statement)	Formatted query results (max 100 rows)	Executes safe, read-only SQL queries. Enforces mutation blocking and applies a default LIMIT to prevent large or unsafe queries.

Table 1: Available tools used by the SQL ReAct Agent

Safety Features Built-in:

- Rejects **INSERT, UPDATE, DELETE, DROP, ALTER**
- Enforces automatic **LIMIT 100** for unbounded selects
- Validates query structure before execution
- Strict read-only database connection

1.4 Example Run and Trace

Using the Gemini 2.5 Flash model, the agent performs explicit step-by-step reasoning. Below is an excerpt from an actual logged execution:

```
1 =====
2 NEW QUERY: How many customers are in the database?
3 =====
4
5 STEP 1
6 THOUGHT: I need to find what tables exist first.
7 ACTION: list_tables{}
8 OBSERVATION: Available tables: customers, orders
9
10 STEP 2
11 THOUGHT: Inspecting the 'customers' table will allow me to count rows.
12 ACTION: describe_table{"table_name": "customers"}
13 OBSERVATION:
14   Columns: id, name, email, age, city
15   Row count: 5
16
17 FINAL ANSWER: There are 5 customers in the database.
```

Listing 4: Sample Test

This structured trace allows comprehensive auditing of the LLM's reasoning and greatly improves agent reliability.

2 Test Results (6 Tests)

All tests were executed on a sample database containing customers and orders tables. The full run details are preserved in test_log.txt and agent_log.txt. A summary of each test is provided below.

2.1 Test-1: Schema Discovery

Query: *What Tables exist?*

Expected: Tables: customers, orders

```
=====
2025-11-21 12:45:11,367 - INFO - NEW QUERY: What tables exist?
2025-11-21 12:45:11,367 - INFO - =====
2025-11-21 12:45:12,935 - INFO - ACTION: list_tables{}
2025-11-21 12:45:12,935 - INFO - OBSERVATION: Available tables: customers, orders
2025-11-21 12:45:12,935 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:45:16,522 - INFO -
FINAL ANSWER: Available tables: customers, orders
2025-11-21 12:45:24,525 - INFO -
=====
```

Figure 1: Working of Query 1

Agent Behaviour:

- Used list_tables
- Returned correct table names

Result: ✓Passed

Schema discovery worked as intended.

2.2 Test-2: Simple Count

Query: *How many orders are there?*

Expected: 6 orders

```
2025-11-21 12:45:24,525 - INFO - NEW QUERY: How many orders are there?
2025-11-21 12:45:24,525 - INFO - =====
2025-11-21 12:45:26,097 - INFO - ACTION: list_tables{}
2025-11-21 12:45:26,097 - INFO - OBSERVATION: Available tables: customers, orders
2025-11-21 12:45:26,097 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:45:31,264 - INFO - ACTION: describe_table{"table_name": "orders"}
2025-11-21 12:45:31,264 - INFO - OBSERVATION: Table: orders
Columns:
- id (INTEGER)
- customer_id (INTEGER)
- product (TEXT)
- amount (DECIMAL(10,2))
- order_date (DATE)
Row count: 6
2025-11-21 12:45:31,264 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:45:34,606 - INFO - 
FINAL ANSWER: There are 6 orders.
```

Figure 2: Working of Query 2

Agent Behaviour:

- Listed Tables
- Described **orders** table
- Executed `SELECT COUNT(*) FROM orders`

Result: ✓Passed

Correctly identified and counted 6 orders.

2.3 Test-3: Aggregation Query

Query: *What is the total amount of all orders*

Expected: 2059.94

```
=====
2025-11-21 12:45:42,614 - INFO - NEW QUERY: What is the total amount of all orders?
2025-11-21 12:45:42,614 - INFO - =====
2025-11-21 12:45:44,114 - INFO - ACTION: list_tables{}
2025-11-21 12:45:44,114 - INFO - OBSERVATION: Available tables: customers, orders
```

Figure 3: Working of Query 3

```
- order_date (DATE)
Row count: 6
2025-11-21 12:45:58,155 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:46:01,822 - INFO - 
FINAL ANSWER: The total amount of all orders is 2059.94.
```

Figure 4: Working of Query 3

Agent Behaviour:

- Listed tables
- described **orders** table
- Executed `SELECT SUM(amount) FROM orders`
- Returned the correct aggregated value

Result: ✓Passed

Correct aggregation logic.

2.4 Test-4: Filtering Query

Query: *How many customers are from Chicago?*

Expected: 1 customer

```

2025-11-21 12:46:09,830 - INFO - NEW QUERY: How many customers are from Chicago?
2025-11-21 12:46:09,830 - INFO - =====
2025-11-21 12:46:11,308 - INFO - ACTION: list_tables{}
2025-11-21 12:46:11,309 - INFO - OBSERVATION: Available tables: customers, orders
2025-11-21 12:46:11,310 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:46:15,213 - INFO - 
STEP 2|
THOUGHT: The user wants to know how many customers are from Chicago. I have identified the 'customers' table. Now I need to
2025-11-21 12:46:15,213 - INFO - ACTION: describe_table{"table_name": "customers"}
2025-11-21 12:46:15,213 - INFO - OBSERVATION: Table: customers
Columns:
- id (INTEGER)
- name (TEXT)
- email (TEXT)
- age (INTEGER)
- city (TEXT)
Row count: 5
2025-11-21 12:46:15,213 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:46:17,601 - WARNING - [Rate limit hit] Waiting 44.4s before retry 1/3...
2025-11-21 12:47:05,281 - INFO - ACTION: query_database{"query": "SELECT COUNT(*) FROM customers WHERE city = 'Chicago'"}
2025-11-21 12:47:05,281 - INFO - OBSERVATION: Columns: COUNT(*)
Returned 1 row(s):
1. (1,)

2025-11-21 12:47:05,281 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:47:08,975 - INFO - 
FINAL ANSWER: There is 1 customer from Chicago.

```

Figure 5: Working of Query 4

Agent Behaviour:

- Listed tables
- Inspected customers table to confirm availability of city column
- Ran `SELECT COUNT(*) FROM customers WHERE city='Chicago'`

Result: ✓Passed

Correct filtering by city.

2.5 Test-5: Customer count

Query: *How many customers are in the database?*

Expected: 5 customers

```

2025-11-21 12:47:16,977 - INFO - NEW QUERY: How many customers are in the database?
2025-11-21 12:47:16,977 - INFO - =====
2025-11-21 12:47:18,354 - INFO - ACTION: list_tables{}
2025-11-21 12:47:18,355 - INFO - OBSERVATION: Available tables: customers, orders
2025-11-21 12:47:18,355 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:47:21,799 - INFO - ACTION: describe_table{"table_name": "customers"}
2025-11-21 12:47:21,801 - INFO - OBSERVATION: Table: customers
Columns:
  - id (INTEGER)
  - name (TEXT)
  - email (TEXT)
  - age (INTEGER)
  - city (TEXT)
Row count: 5
2025-11-21 12:47:21,801 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:47:25,234 - INFO - 
FINAL ANSWER: There are 5 customers in the database.

```

Figure 6: Working of Query 5

Agent Behaviour:

- Used schema exploration
- Validated that customers exists
- Returned the row count based on schema+query

Result: ✓Passed

Counted rows accurately.

2.6 Test-6: Error Recovery Case

Query: *How many employees are there?*

Expected: An error message indicating that the table does not exist.

```

2025-11-21 12:47:33,245 - INFO - NEW QUERY: How many employees are there?
2025-11-21 12:47:33,245 - INFO - =====
2025-11-21 12:47:35,274 - INFO - ACTION: list_tables{}
2025-11-21 12:47:35,276 - INFO - OBSERVATION: Available tables: customers, orders
2025-11-21 12:47:35,276 - INFO - [Rate limit] Waiting 2.0s before next request...
2025-11-21 12:47:38,623 - INFO - 
FINAL ANSWER: I cannot find information about employees. The available tables are `customers` and `orders`.

```

Figure 7: Working of Query 6

Agent Behaviour:

- Checked available tables
- Identified missing table
- Returned: *"I cannot find information about employees..."*

Result: ✓Passed

Correct error handling and recovery.

3 Reflection

This project highlights the strengths and limitations of building a ReAct-style agent without any third-party agent frameworks.

3.1 Design Tradeoffs

- **Explainability over complexity:** The agent provides full reasoning logs, making it excellent for education and debugging. However, this introduces additional latency due to multi-step LLM calls.
- **Strict safety vs. flexibility:** By permitting only SELECT queries and sanitizing all SQL input, the system becomes highly safe. But this restricts operations such as multi-statement queries or temporary views.
- **Lightweight dependencies:** Using only the Gemini API keeps the system minimal and transparent. This means the developer has full control, but also full responsibility for error handling and agent behavior.

3.2 Observed Failure Modes

- **Occasional hallucination of column names** (when schema inspection is skipped)
- **Tool repetition loops** if the agent fails to converge without step limits
- **Ambiguous prompts** (e.g., “recent orders”) cannot be resolved without explicit temporal logic
- **Rate limit interruptions** occasionally trigger automatic backoff cycles

3.3 Future Enhancements

- Schema-based query validation before execution
- In-memory conversation history to avoid repeated table discovery
- Support for PostgreSQL, MySQL, or cloud-hosted databases
- Automatic query refinement when SQL errors occur
- Semantic caching to reduce API calls for repeated queries

4 Conclusion

This project successfully demonstrates that a fully custom ReAct-based SQL agent can be implemented from scratch while maintaining:

- Strong safety guarantees
- Transparent multi-step reasoning
- Reliable performance across diverse query types
- Complete error recovery and rate-limit handling

By combining structured reasoning patterns with LLM capabilities, the system offers a powerful yet interpretable interface between natural language and relational databases. It establishes a strong foundation for future intelligent database assistants capable of more sophisticated SQL reasoning, multi-database support, and adaptive learning.